

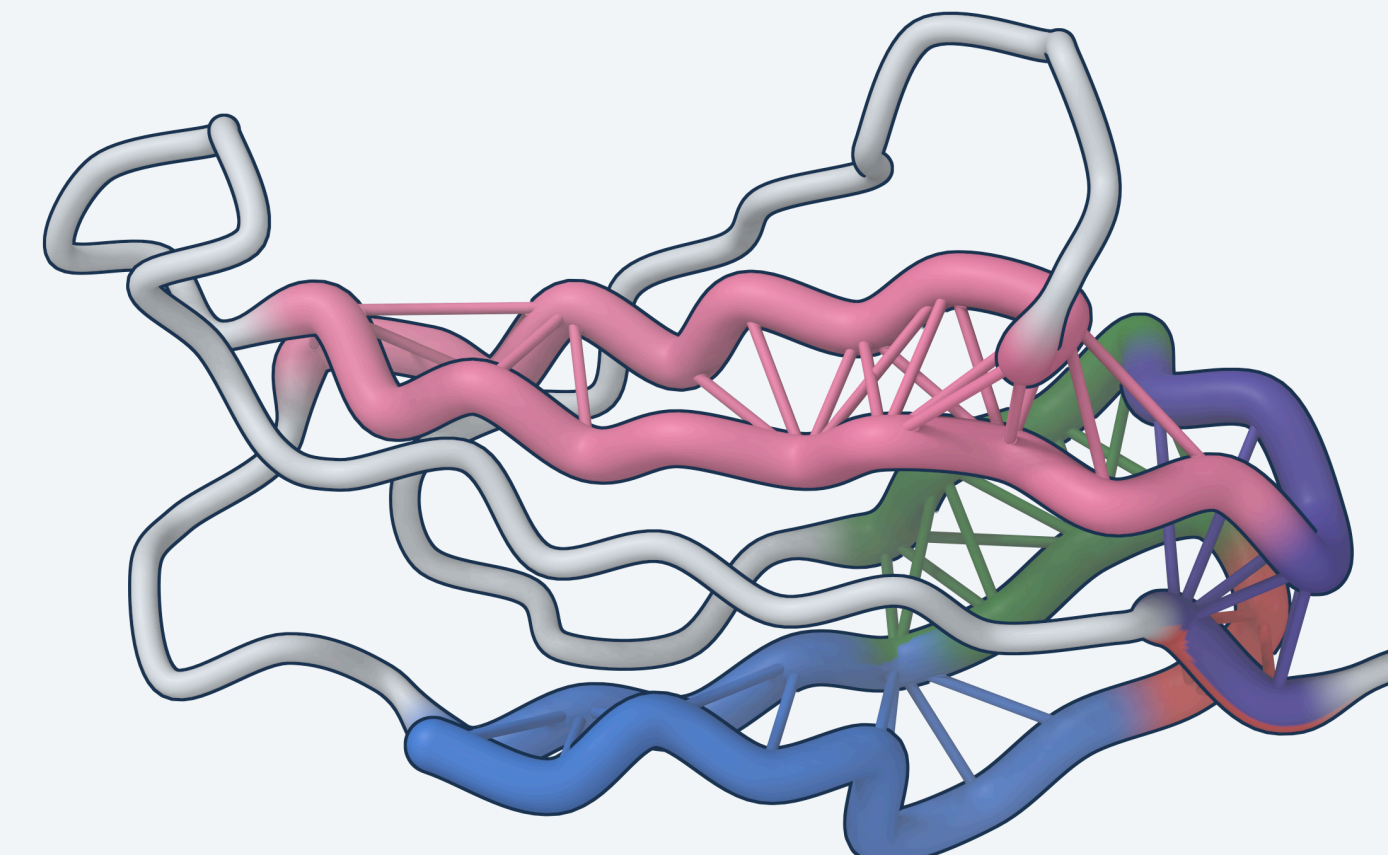
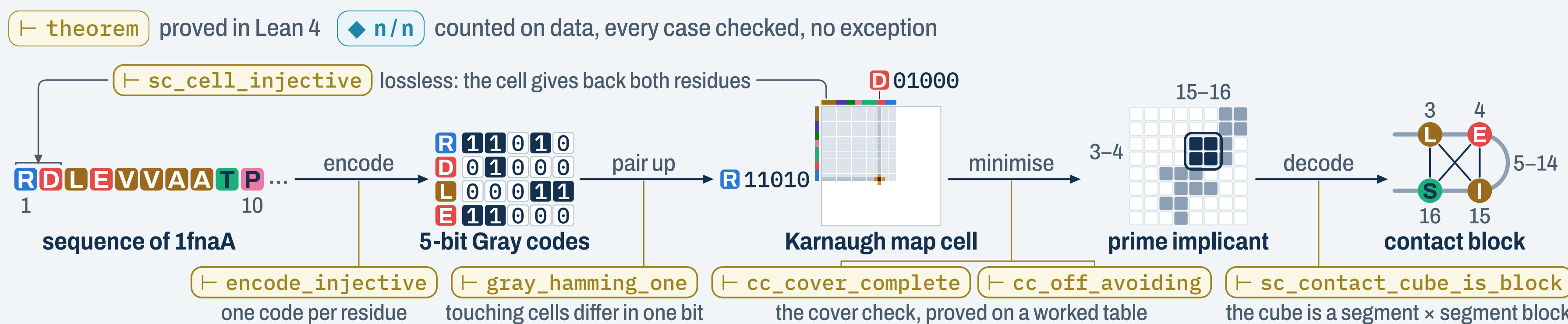
Is KMAP Possible? Formal Verification of Karnaugh Map Encoding for Biological Sequences in Lean 4

Shuvam Banerji Seal, Susmita Roy*, Dwaipayan Roy
Indian Institute of Science Education and Research Kolkata, India
*susmita.roy@iiserkol.ac.in

000 The finding

We formalised in Lean 4 the mathematics that turns DNA, RNA and protein sequences into Karnaugh maps: 131 machine-checked theorems, none left unproved. On this foundation every structure becomes an exact Boolean circuit whose rules hold without exception, for proteins, a viral species and folding simulations.

Gray-coded grids of sequences, from Petoukhov's genetic Karnaugh matrices¹ to chaos-game maps (FCGR)^{2,3}, are usually taken on trust. Here the encodings are theorems and every circuit and rule is checked on every case.

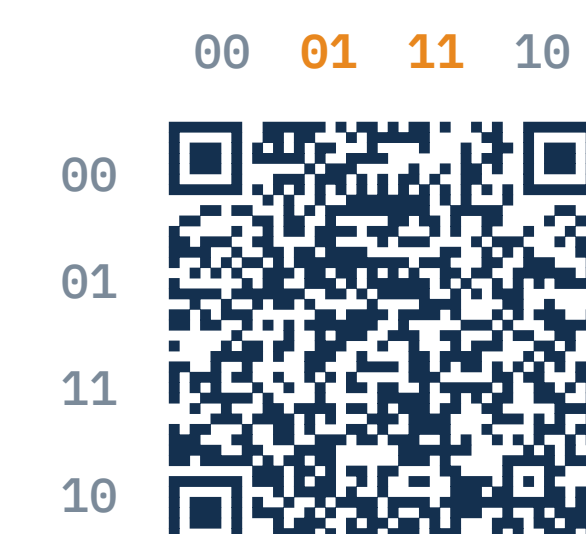


Fibronectin type-III domain (PSICOV 1fnaA, 91 residues). Its exact contact circuit, read as rules: block rules that continue one another are strand pairs, one colour each.

Strand pairs, found as blocks

- 25–34 with 61–70 6 blocks
- 1–8 with 14–18 4 blocks
- 11–14 with 53–56 3 blocks
- 57–60 with 89–90 2 blocks
- 9–10 with 89–90 1 block

Turn any protein into a circuit

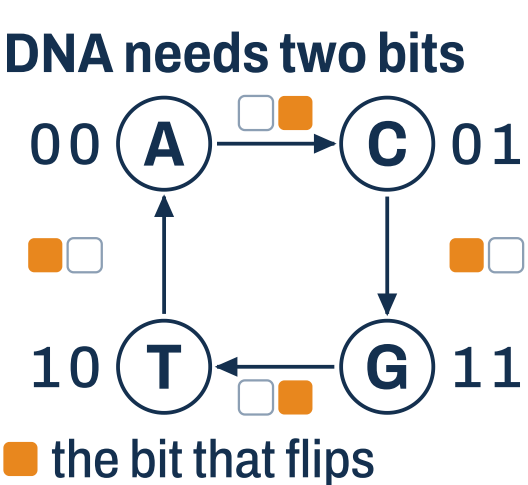


Any PDB entry, step by step, to its rules and inferences.

001 A code that changes one bit at a time

Binary counting can flip many bits at once: 0111 → 1000 flips four. The reflected Gray code $g(n) = n \oplus (n \gg 1)$ visits every number once and changes exactly one bit per step. DNA needs two bits, and A C G T is itself a Gray cycle.

n	binary	flips	Gray $g(n)$	flips
0	000		000	
1	001	1	001	1
2	010	2	011	1
3	011	1	010	1
4	100	3	110	1
5	101	1	111	1
6	110	2	101	1
7	111	1	100	1



⊢ gray_hamming_one ⊢ gray_injective ⊢ gray_cycle_AC...TA

011 A Karnaugh map⁴ is a folded truth table

Order a grid's rows and columns by the Gray code and touching cells differ in one variable, even across the edges, which wrap like a torus. Put a k -mer's first letters on the rows and the rest on the columns: each 2-mer owns one cell of a 4×4 map, each 4-mer one of a 16×16 map.

first base	second base	
A 00	A 00 C 01 G 11 T 10	
A 00	AA AC AG AT	00 10
C 01	CA CC CG CT	01 11
G 11	GA GC GG GT	11 10
T 10	TA TC TG TT	

One cell per k -mer
 $4 \times$ the cells per base

$k = 1$ $k = 2$ $k = 3$ $k = 4$
 2×2 4×4 8×8 16×16

⊢ dinucEncode_injective ⊢ tetranucEncode_injective ⊢ gray_code_preserves_adjacency

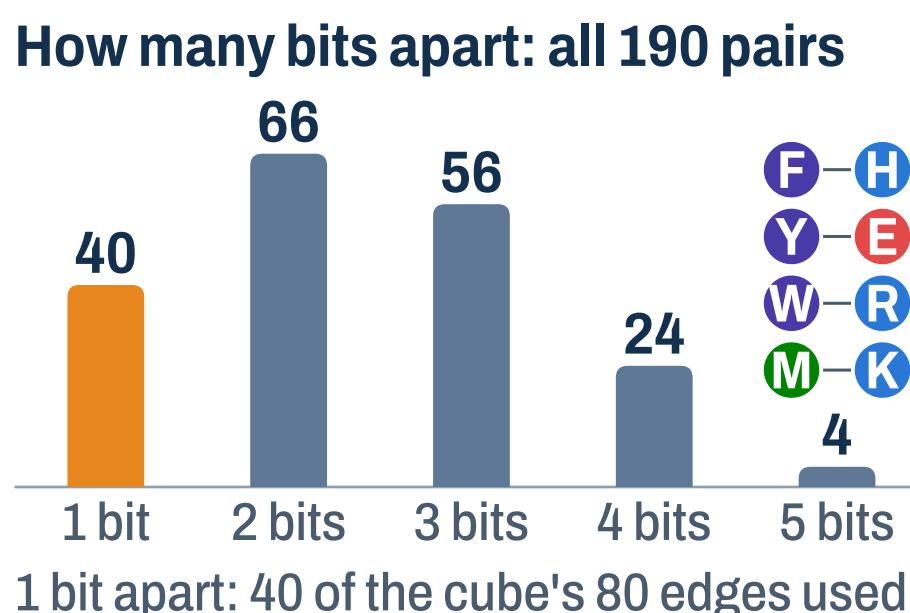
010 Amino acids on a five-dimensional cube

Proteins need five bits: 32 codewords for 20 residues. Numbering residues group by group before Gray-coding keeps chemistry close. All 190 pairwise distances are proved: 40 pairs sit one bit apart, only four sit five apart, and the code uses exactly half of the cube's 80 edges.

Flattened into a Karnaugh map: touching cells differ in one bit, and the edges wrap

first 2 bits	000	001	011	010	110	111	101	100
last 3 bits	00	01	11	10	00	01	11	10
00	A	V	L	I	F	Y	W	M
01	D	Q	N	T	S	G	P	C
11	E	H	K	R				
10								

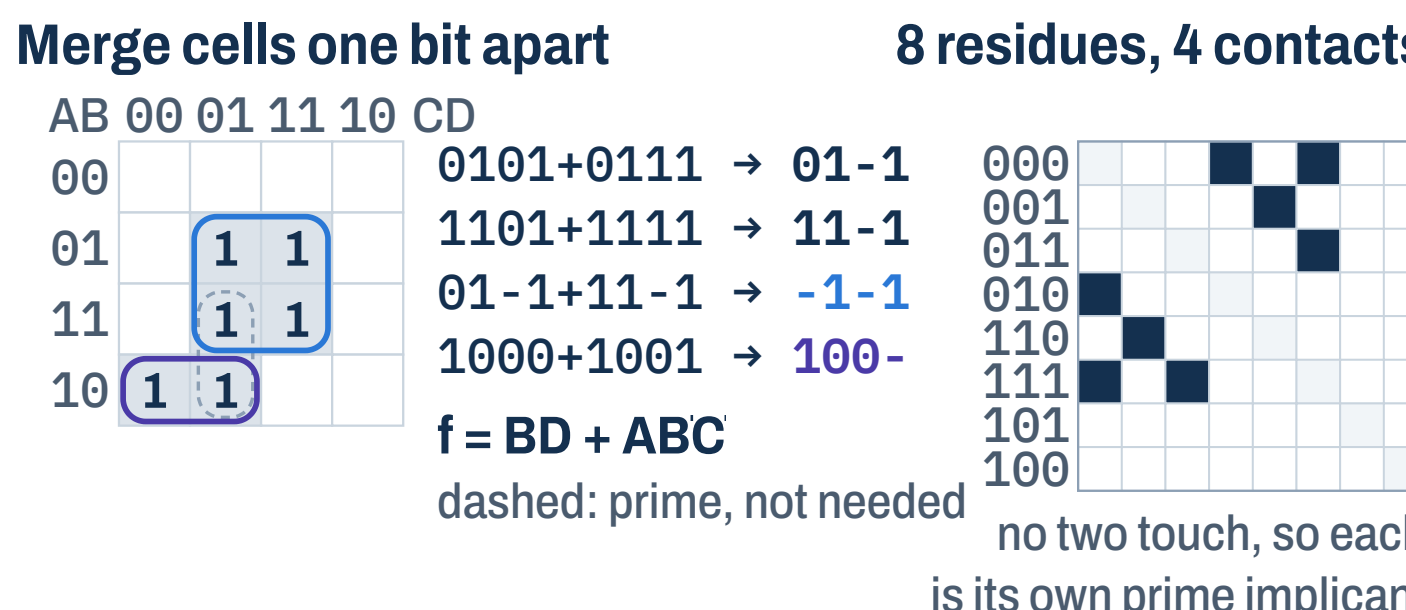
- aliphatic hydrophobic
- aromatic
- sulfur-containing
- structure-breaking
- polar uncharged
- negatively charged
- positively charged
- unused codeword (12)



⊢ encode_injective ⊢ unorderedDistanceDistribution ⊢ max_distance_FH...MK
 ⊢ encoding_edge_coverage

110 Contacts as a Boolean function

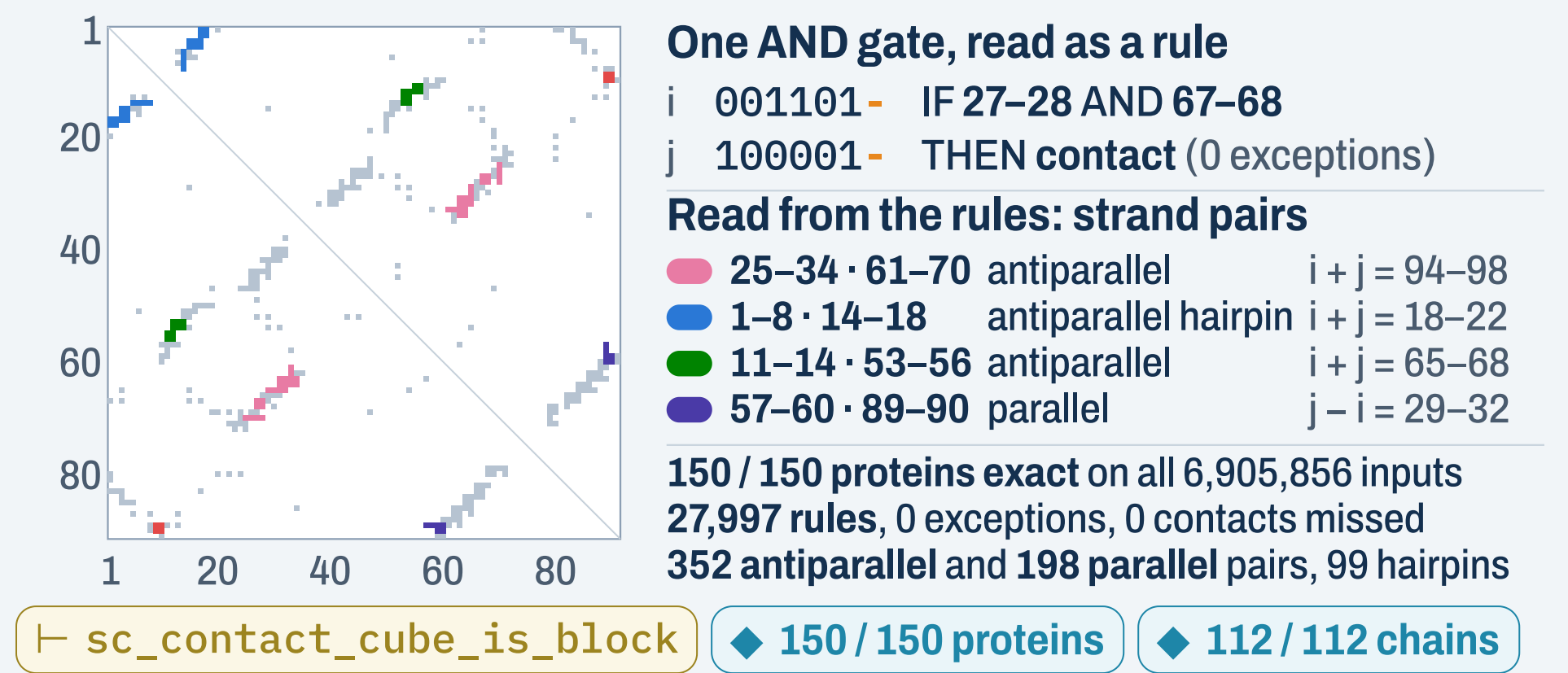
A contact map is a Boolean function: $C(i, j) = 1$ when residues i and j lie within 8 Å. Quine–McCluskey⁵ merges 1-cells one bit apart into ever larger cubes; the cubes that cannot grow are the prime implicants, and a cover of them is a two-level circuit equal to the map. For an 8-residue example Lean proves the map symmetric and that no two of its contacts can merge.



⊢ contactMap8_symmetric ⊢ contactMap_no_groupable_minterms ⊢ cc_cover_complete

100 Rules with no exception, from exact circuits

Each PSICOV⁷ protein becomes a circuit equal to its contact map. Its AND gates are rules; block rules that continue one another give strand pairs, direction and register, by counting.



One species: in 1,299 SARS-CoV-2 Spike sequences, exact circuits list which residue pairs occur at co-evolving positions and which never do. No sequence breaks a rule.

positions 371 × 376	positions 405 × 408	positions 486 × 493
1,294 sequences	1,289 sequences	1,297 sequences
A376 T376	R408 S408	Q493 R493
F371 970 0	D405 319 0	F486 20 1,173
L371 0 311	N405 12 958	P486 2 0
S371 1 12		V486 102 0
F371 ⇒ A376 in 970 of 970	D405 ⇒ R408 in 319 of 319	V486 ⇒ Q493 in 102 of 102
L371 ⇒ T376 in 311 of 311	S408 ⇒ N405 in 958 of 958	R493 ⇒ F486 in 1,173 of 1,173
10 position pairs · 41 allowed and 35 forbidden residue combinations · 23 implications		
0 violations: every one of the 1,299 sequences obeys every rule		

◆ 1,299 / 1,299 sequences obey every rule ◆ unique minimum covers

111 How Lean checks a claim

In Lean 4⁶ a statement is a type and a proof is a program checked by a small kernel. For finite claims, native_decide evaluates every case in compiled code, up to 2²⁰ in one theorem, and so also trusts Lean's compiler. No proof uses sorry or adds an axiom of its own.

```
-- Gray code is additive over GF(2)
theorem sc_gray_gf2_linear :
  ∀ x y : Fin 32,
    scGray (x.val ^^^ y.val)
      = scGray x.val ^^^ scGray y.val
:= by native_decide
```

◆ axiom audit: 131 theorems, 0 sorry ◆ Python ↔ Lean bridge: 156/156 checks

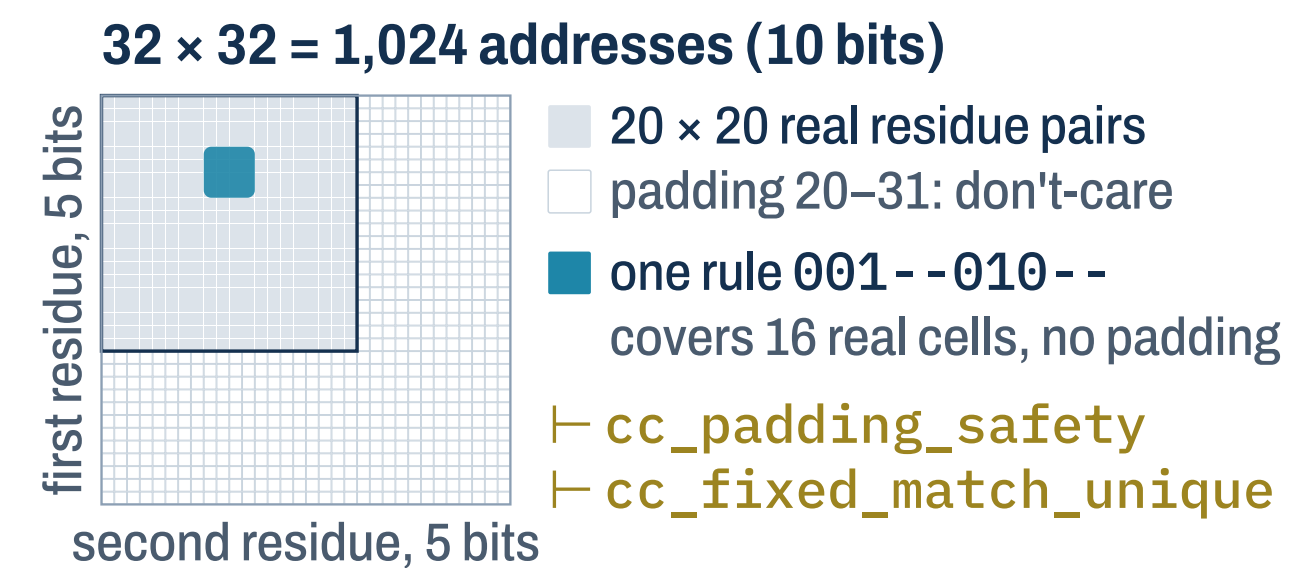
	native_decide	standard axioms only	no axiom
KmapProofs	16		
KmerIndexing	35		
AminoAcidEncoding	27		
ContactMapCompleteness	20		
KmapEncodingEquiv	8		
ContactCircuits	12		
SequenceCircuits	13		
131 theorems			

101 What the proofs guarantee

Exact rules on padded maps

Twenty residues need five bits per axis, so a 20×20 residue-pair map is padded to 32×32 with 10-bit addresses. Lean proves that a fully fixed rule matches exactly its own cell and that any rule on real residues fires only inside the 20×20 block: padding never enters a result.

⊢ cc_padding_safety ⊢ cc_fixed_match_unique ⊢ cc_off_avoiding



Segments hold under either numbering

With its low position bits free, a cube is an unbroken chain segment. Lean proves this for Gray-coded positions as well as plain binary: the two numberings name different segments, never broken ones.

⊢ sc_gray_cube_is_also_interval ⊢ sc_gray_relabels_segments

Same cube, still a segment, different residues

Plain binary cube 10 - - → residues 8–11

00	00	00	00	01	01	01	01	10	10	10	10	11	11	11	11
00	01	10	11	00	01	10	11	00	01	10	11	00	01	10	11
01	01	01	01	01	01	01	01	01	01	01	01	01	01	01	01
01	10	10	10	10	10	10	10	10	10	10	10	10	10	10	10
10	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
10	01	11	10	10	11	01	00	00	01	11	10	10	11	00	00

Gray-coded cube 10 - - → residues 12–15

What the encoding can and cannot change

Since $g(x \oplus y) = g(x) \oplus g(y)$, a Gray-ordered map is a lexicographic (FCGR) map with its cells permuted: every statistic of the cell values is identical. The code changes only distances between letters: of the 400 ordered amino-acid pairs, exactly 80 are one bit apart.

⊢ sc_gray_gf2_linear ⊢ cc_he_dist_counts ◆ 120 / 120 maps: same cell values

Lexicographic (FCGR) Gray Same 16 counts, moved

A C G T	i → g(i)	A C T G	CGCGAATTCGCG
A 1 1 1	00 → 00	A 1 1 1	(1BNA), address order
C 1 1 1	01 → 01	C 1 1 1	◆ cophenetic r
G 1 1 1	10 → 10	G 1 1 1	0.976747 = 0.976747
T 1 1 1	11 → 11	T 1 1 1	

What Gray changes: code distance d, 400 residue pairs

d	0	1	2	3	4	5
count	20	80	132	112	48	8
total	80 / 400					

pairs one bit apart

Rules that hold in every frame of a folding run

In structure-based-model⁸ trajectories every native contact is formed or broken in every frame. Rules “whenever a is formed, so is t ” that no frame breaks join neighbouring contacts; in ten long 8,000-frame runs, all of them do.

◆ 9,986 rules, 0 counterexample frames ◆ 479 trajectories, 48,385 frames

Crambin, 8,000 frames

a = contact 4–44, t = 4–43	t formed	t broken
a formed	5,429	0
a broken	1,258	1,313

9,986 rules hold in every frame of 479 folding runs like a ⇒ t: a formed never without t

3,966 join neighbouring contacts 33 / 33 in the long runs

1. S. V. Petoukhov, *BioSystems* **2024**, 246, 105349.
 2. H. J. Jeffrey, *Nucleic Acids Res.* **1990**, 18, 2163–2170.
 3. J. S. Almeida, S. Vinga, *BMC Bioinformatics* **2009**, 10, 100.

4. M. Karnaugh, *Trans. AIEE* **1953**, 72, 593–599.
 5. W. V. Quine, *Am. Math. Mon.* **1952**, 59, 521–531; E. J. McCluskey, *Bell Syst. Tech. J.* **1956**, 35, 1417–1444.

6. L. de Moura, S. Ullrich, *CADE-28*, LNCS 12699, **2021**, 625–635.
 7. D. T. Jones et al., *Bioinformatics* **2012**, 28, 184–190.
 8. J. K. Noel et al., *PLoS Comput. Biol.* **2016**, 12, e1004794.